
Concurrencia y Paralelismo

Bloque I: Concurrencia

Julio 2023

1. Poetas [2.5 puntos]

Un grupo de N poetas se sienta en una mesa formando una fila, numerados de 0 a $N-1$. Disponen de un único bolígrafo, por lo que solo una persona puede estar escribiendo a la vez. El número de quien tiene el bolígrafo está indicado por el campo `pen_holder`, y el número de a quien le toca escribir por `current_poet`.

Cuando la persona indicada por `current_poet` recibe el bolígrafo escribe durante un tiempo, representado por una llamada a la función `write()`, que debe hacerse sin ningún `mútex` bloqueado. Cuando finaliza, escoge a la siguiente persona que recibirá el bolígrafo generando un número aleatorio entre 0 y $N-1$, pero garantizando que no es ella misma.

Como están sentados en la mesa formando una fila, para hacer llegar el bolígrafo hasta esa persona tiene que irse pasando por todas las personas que estén entre ellas. Por ejemplo, para pasar el bolígrafo de la persona 5 a la 8, la 5 se lo pasaría a la 6, la 6 a la 7, y la 7 a la 8. Hay que tener en cuenta que el bolígrafo puede tener que ir hacia el final o hacia el principio de la mesa.

Cuando una de las personas no tenga que escribir ni pasar el bolígrafo deberá estar esperando. No deben usarse esperas activas, ni despertar a más de una persona a la vez.

Defina la función `poet` de tal manera que se comporte de la forma descrita.

```
#define N ...

void write(); // Call this function to simulate the writing
int rand();  // generate a random number between 0 and RAND_MAX

typedef struct {
    int current_poet; // starts with a valid value from 0 to N-1
    int pen_holder    // which poet has the pen right now.
    ...               // starts with the same value as current_poet
} poet_group;

typedef struct {
    int id; // poet number
    poet_group *group; // shared structure
    ...
} poet_info;

void poet(poet_info *inf) {

}
```

```
int mtx_lock(mtx_t *mtx);
int mtx_trylock(mtx_t *mtx);
int mtx_unlock(mtx_t *mtx);
int cnd_wait(cnd_t *cond, mtx_t *mtx);
int cnd_signal(cnd_t *cond);
int cnd_broadcast(cnd_t *cond);
```

Solución:

```
#define N ...

void write(); // Call this function to simulate the writing
int rand();  // generate a random number between 0 and RAND_MAX

typedef struct {
    int current_poet; // starts with a valid value from 0 to N-1
    int pen_holder    // which poet has the pen right now.
                    // starts with the same value as current_poet
    mtx_t lock;
    cnd_t waiting[N];
} poet_group;

typedef struct {
    int id; // poet number
    poet_group *group; // shared structure
} poet_info;

void poet(poet_info *inf) {
    while(true) {
        mtx_lock(&inf->group->lock);
        while(inf->group->pen_holder != inf->id)
            cnd_wait(&inf->group->waiting[inf->id], &inf->group->lock);
        if(inf->group->current_poet == inf->group->id) {
            mtx_unlock(inf->group->lock);
            write();
            mtx_lock(inf->group->lock);
            while(inf->group->current_poet == inf->id)
                inf->group->current_poet = rand() % N;
        }
        if(inf->group->current_poet > inf->id)
            inf->group->pen_holder++;
        else
            inf->group->pen_holder--;
        mtx_signal(&inf->group->waiting[inf->group->pen_holder]);
        mtx_unlock(&inf->group->lock);
    }
}
```

2. Pool de recursos [2.5 puntos]

Un grupo de threads necesita usar una unidad de un recurso del cual hay unidades limitadas, pero que no se consume al usarlo. Para manejar esos recursos se utiliza una pila almacenada en un array con un número que marca la cima.

Cuando un thread necesita hacer una operación, tiene que comprobar si hay alguno disponible. Si es así, lo retira de la estructura, lo usa, y lo vuelve a insertar para que los siguiente puedan utilizarlo. Si no hay ninguno libre debería esperar a que lo haya sin usar esperas activas.

Implemente la función `worker` con el comportamiento descrito.

```
typedef struct {
    resource *pool[...];
    int top; // top of the resource stack
    ...
} resource_pool;

void worker(resource_pool *p) {
    resource *r;

    ...
    use(r);
    ...
}
```

Solución

```
typedef struct {
    resource *pool[...];
    int top; // top of the resource stack
    mtx_t lock;
    cnd_t empty;
} resource_pool;

void worker(resource_pool *p) {
    resource *r;

    mtx_lock(&p->lock);
    while (p->top == -1)
        cnd_wait(&p->empty, &p->lock);
    r = p->pool[p->top];
    p->top--;
    mtx_unlock(&p->lock);

    use(r);

    mtx_lock(&p->lock);
    p->top++;
    p->pool[p->top] = r;
    if(p->top == 0)
        mtx_broadcast(&p->empty);
    mtx_unlock(&p->lock);
}
```

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

Convocatoria extraordinaria

11 de julio de 2023

AVISO: *No mezcléis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregan por separado.*

- [2,5p] 1. **Conceptos de paralelismo.** La policía local de tráfico de una determinada ciudad quiere automatizar la regulación de los semáforos. Para ello se quiere desarrollar un software que analice cada noche los vídeos de diferentes cámaras de tráfico y, en base a lo observado en ellos, la propia aplicación modificará automáticamente los tiempos de cada semáforo. El análisis de todos los vídeos es muy costoso, y una implementación secuencial tradicional no terminaría a tiempo para modificar los semáforos de cara al día siguiente. Concretamente, con el número de cámaras actuales se generan miles de vídeos al día y se necesitarían 20 horas para que un programa secuencial analizase todos los vídeos, de las cuales solo 5 minutos se deben al algoritmo de toma de decisiones (el resto de tiempo es análisis de vídeos uno tras otro). Dicho algoritmo no puede empezar hasta que todos los vídeos hayan sido analizados. Por eso se desea realizar una implementación paralela de este software.
- (a) Asume que cuando arranca el programa todos los vídeos están ya cargados en el sistema (se han ido cargando durante el día) y que el algoritmo de toma de decisiones no se puede paralelizar. ¿Cuál es la aceleración máxima que se puede obtener?[0,5p]
 - (b) Teniendo en cuenta los siguientes datos, indica DE FORMA RAZONADA cuál sería la mejor descomposición y asignación de tareas [0,5p]
 - Cada vídeo individual no se puede dividir entre varios procesos.
 - El tiempo de análisis de cada vídeo puede variar mucho, ya que influye tanto su duración (no todos duran lo mismo) como el número de calles y vehículos que están involucrados.
 - No se puede saber a priori el tiempo de análisis de un cierto vídeo.
 - Solo un servidor (llamado “de almacenamiento”) tiene contacto con las cámaras, y se usará siempre exclusivamente para alojar los vídeos y suministrarlos al resto de servidores (llamados “de análisis”).
 - Los servidores de análisis no están conectados entre sí, solo con el de almacenamiento.
 - (c) Para el primer año de la experiencia se han comprado 9 servidores de análisis. En una determinada noche el tiempo medio de análisis en esos servidores ha sido de 3 horas, con un máximo de 5 horas para el que más tuvo que analizar, y un mínimo de 2 horas para el que menos. Teniendo en cuenta que tanto el tiempo de envío de los vídeos desde el servidor de almacenamiento como el de la recolección de los datos de análisis para luego aplicar el algoritmo de toma de decisiones es despreciable, calcula DE FORMA RAZONADA: [1p]
 - El tiempo paralelo.
 - La aceleración.
 - La eficiencia.

- El coste.
- La sobrecarga.

(d) Supón que adoptamos otra aproximación donde el servidor de almacenamiento también puede realizar análisis. Además, se puede hacer un preprocesado con bajo coste computacional de los vídeos para estimar a priori cuánto se tardará en analizar cada uno. ¿Cambiarías la descomposición o la asignación indicada en el apartado b)? [0,5p]

[2,5p] 2. **Diseño de algoritmos paralelos.** Se quiere paralelizar el siguiente código, que trabaja sobre una matriz A de dos dimensiones (M filas y N columnas) utilizando una determinada función $func$:

```
int *A = (int *)malloc(sizeof(int) * M * N);
int *B = (int *)malloc(sizeof(int) * M * N);
int aux, max = 0;
leeDeDisco(A);

for(int i = 0; i < M; i++)
    for(int j = 1; j < N-1; j++){
        aux = func(A[i*N+j], A[i*N+j-1], A[i*N+j+1]);
        B[i*N+j] = aux;
        if(max < aux)
            max = aux;
    }

escribeResultado(B, max);
```

Solo uno de los procesos tiene acceso a disco y pantalla. Por tanto, este será el único proceso que puede ejecutar las funciones `leeDeDisco` y `escribeResultado`. Además, tanto M como N son siempre múltiplos del número de procesos. Contesta DE FORMA RAZONADA las siguientes preguntas:

- Escribe un código MPI que permita ejecutar de forma paralela este programa, usando colectivas siempre que se pueda (ver la tabla del final con la sintaxis de las rutinas MPI). [1p]
- Supón un caso particular en el que se usan 10 procesos MPI para una ejecución con $M = 5000$ y $N = 2000$. El tiempo de la función $func()$ es siempre un segundo, y el tiempo de una determinada función colectiva es $D * 10^{-3}$ segundos, siendo D el número de elementos totales enviados (es decir, la suma de los elementos enviados a cada proceso). Indica cuál sería la acalerción obtenida por el código que presentas en el apartado anterior si el tiempo del resto de actividades es despreciable. [0,75]
- En un programa alternativo la función $func()$ depende de cinco parámetros, ya que también se introducen $A[(i+1)*N+j]$, $A[(i-1)*N+j]$. Explica, de palabra, cómo cambiarías la distribución de los datos (tanto si se tienen que mandar más o menos datos como si tienes que usar rutinas de comunicación diferentes). ¿Se debe modificar la recolección de datos? [0,75]

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
             int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
              void *recvbuf, int recvcnt, MPI_Datatype recvtype,
              int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
              void *recvbuf, int recvcnt, MPI_Datatype recvtype,
              int root, MPI_Comm comm)
int MPI_Allreduce(void *sendbuf, void *recvbuf, int count,
                 MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op,
               int root, MPI_Comm comm)
```

SOLUCIÓN

1. (a) Por la Ley de Amdahl la aceleración máxima es:

$$\frac{T_s + T_p}{T_s}$$

En este caso, la parte intrínsecamente secuencial es 5 minutos, sobre un total de 20 horas (1200 minutos). Por tanto, la aceleración máxima es:

$$\frac{1200}{5} = 240$$

- (b) La parte del programa a paralelizar es el análisis de los vídeos, a los que se les aplica la misma función. Se indica que los vídeos no se pueden subdividir, por lo tanto el análisis de cada vídeo es la tarea más pequeña que se puede hacer. Esto es una descomposición de dominio.

La única asignación que se puede aplicar es una dinámica maestro-esclavo, donde el servidor de almacenamiento es el maestro. Una estática no tiene sentido por dos razones.

- i. Tenemos un servidor que no realiza análisis.
- ii. El tiempo de análisis para cada vídeo puede variar mucho, y además el coste computacional no tiene nada que ver con el orden del vídeo (no se ganaría rendimiento por usar una asignación estática cíclica).

Una asignación dinámica completamente descentralizada tampoco se puede realizar porque los servidores de análisis no están conectados entre sí.

- (c) • Para el tiempo paralelo debemos quedarnos con el servidor que más tarda (5 horas o 300 minutos) y hay que sumarle los 5 minutos secuenciales del algoritmo de toma de decisiones: 305 minutos.
- La aceleración será:

$$\frac{1200}{305} = 3,93$$

- La eficiencia paralela es:

$$\frac{3,93}{10} = 0,393$$

- El coste es:

$$10 * 305 = 3050$$

- La sobrecarga es:

$$3005 - 1200 = 1850$$

NOTA: Si se usa como tiempo secuencial la media de análisis de vídeos (3 horas) multiplicado por el número de servidores de análisis (9) más los cinco minutos de decisión también es válido.

- (d) En este caso sí tendría sentido una distribución estática, que nos permitiría utilizar también el servidor de almacenamiento para hacer trabajo efectivo de análisis. Como se puede estimar a priori el tiempo que se va a necesitar con cada vídeo, habría que intentar asignar vídeos con una carga similar a todos los procesos.
2. (a) Se aplica una descomposición de dominio con una asignación estática bloque por filas ya que las dependencias están dentro de las filas.


```

int *A, *B, *myA, *myB;
int aux, max, mymax = 0;
myA = (int *) malloc(sizeof(int) * M/numP * N);
myB = (int *) malloc(sizeof(int) * M/numP * N);

if(!rank) {
    A = (int *) malloc(sizeof(int) * M * N);
    B = (int *) malloc(sizeof(int) * M * N);
    leeDeDisco(A);
}

MPI_Scatter(A, M/numP * N, MPI_INT, myA, M/numP * N,
            MPI_INT, 0, MPI_COMM_WORLD);

for(int i = 0; i < M/numP; i++)
    for(int j = 1; j < N-1; j++){
        aux = func(myA[i*N+j], myA[i*N+j-1], myA[i*N+j+1]);
        myB[i*N+j] = aux;
        if(mymax < aux)
            mymax = aux;
    }

MPI_Gather(myB, M/numP*N, MPI_INT, B, M/numP*N,
            MPI_INT, 0, MPI_COMM_WORLD);
MPI_Reduce(&mymax, &max, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);

if(!rank) {
    escribeResultado(B, max);
}

```

- (b) El tiempo secuencial será el de ejecutar la función sobre todos los elementos: $5000 * 2000 * 1seg = 10000000seg$

El tiempo paralelo será la suma del tiempo de computación de cada proceso ($\frac{5000}{10} * 2000 * 1seg = 1000000seg$) y el de comunicaciones. El tiempo de comunicaciones es la suma de:

- Tiempo del scatter: $9 * \frac{5000}{10} * 2000 * 10^{-3} = 9000seg$ (también se dará por válido si entienden que el raíz se manda a sí mismo, lo que implica multiplicar por 10 en lugar de por 9).
- Tiempo del gather con el mismo razonamiento que el scatter: $9000seg$
- Tiempo del reduce: $9 * 1 * 10^{-3} = 0,009$ (también se dará por válido si entienden que el raíz se manda a sí mismo, lo que implica multiplicar por 10 en lugar de por 9)

Esto implica que el tiempo de comunicaciones es más o menos $18000seg$ (el tiempo del reduce es despreciable comparado con los otros). Por tanto, el tiempo paralelo es $1018000seg$ y la aceleración:

$$\frac{10000000}{1018000} = 9,82$$

- (c) En este caso para calcular la función en una posición se necesitan también los vecinos de arriba y abajo. Eso implica que repartir solo las filas A sobre las que va a calcular cada proceso no es suficiente. Se deben enviar también la fila anterior y la posterior al bloque. Por tanto, las filas que queden en el límite de los bloques estarán duplicadas en dos procesos. Eso implica que no se puede usar la colectiva *scatter* para el reparto, sino que habrá que hacer el reparto con rutinas *send/recv*.

La recolección de datos con *gather* y *reduce* no se ve alterada porque cada proceso sigue calculando la misma porción de la matriz B .